Git para Desarrollo Web

Fundamentos y Flujo de Trabajo Práctico

`init`	`commit`	`branch`	`merge`
Inicializar	Guardar	Crear Ramas	Fusionar

¿Qué es el Control de Versiones?

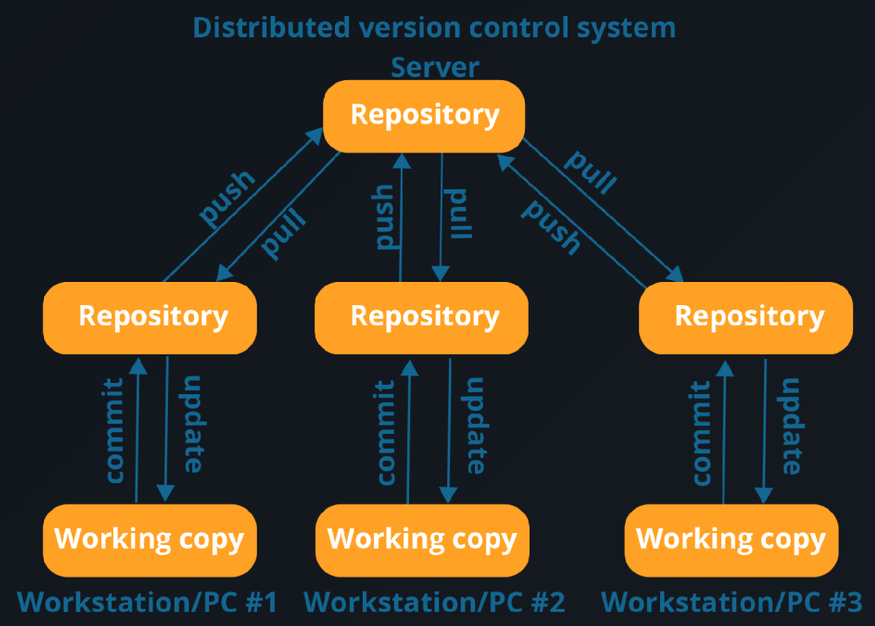
Sistema que registra cambios en archivos a lo largo del tiempo, permitiendo recuperar versiones específicas

Tipos de Sistemas

Locales
Copias manuales en directorios

Centralizados (CVCS)
Servidor único • CVS, Subversion

Distribuidos (DVCS)
Réplicas completas • Git, Mercurial



Instalación de Git

Linux

Ubuntu / Debian

```
sudo apt-get install git
```

Fedora

```
sudo yum install git
```

macOS

Opción 1

Xcode Command Line Tools

Opción 2

Instalador desde git-scm.com

Windows

Opción 1

Instalador oficial
git-scm.com/download/win

Opción 2

GitHub Desktop (GUI)

Verificar instalación: `git --version`

Configuración Inicial (IMPORTANTE)

Necesario para registrar quién hace los commits

Configurar nombre:

```
git config --global user.name "Tu Nombre"
```

Configurar email:

```
git config --global user.email
"tu@email.com"
```

Verificar configuración: `git config --list`

Conceptos Fundamentales - Los Tres Estados

Working Directory

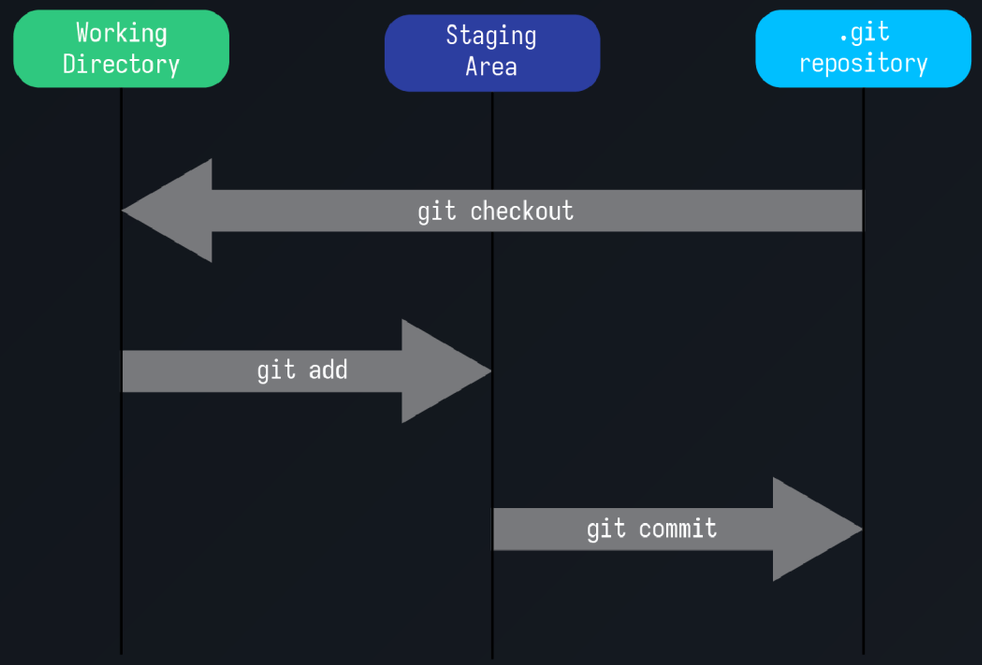
Donde editas archivos • Estado "normal" del proyecto

Staging Area

Área de preparación • Información del próximo commit

Repository

Donde se almacenan los commits permanentemente



`Working` → `git add` → `Staging` → `git commit` → `Repository`

Conceptos: Repositorio y Commit

Repositorio

Base de datos que contiene todo el historial del proyecto

`git init` Crea un repositorio nuevo

`.git` Carpeta con toda la información

Commit

Snapshot del proyecto en un momento específico

- Identificador único (hash SHA-1)
- Metadatos: autor, fecha, mensaje
- Es inmutable (no se puede modificar)

Mensajes de Commit Efectivos

Explican QUÉ se cambió y POR QUÉ

Formato recomendado:

`tipo: descripción clara`

Tipos comunes: feat, fix, docs, style, refactor

`feat:`	`Agregar sistema de login` Nueva funcionalidad
`fix:`	`Corregir error en validación` Corrección de bug
`docs:`	`Actualizar README` Documentación

Consejo: Escribe mensajes que expliquen el cambio como si le hablaras a otro desarrollador

Conceptos: Branch

¿Qué es una rama?

Punteros móviles a commits

Permiten líneas de desarrollo independientes

NO son copias de código

HEAD

Puntero especial que indica la rama actual

Crear rama

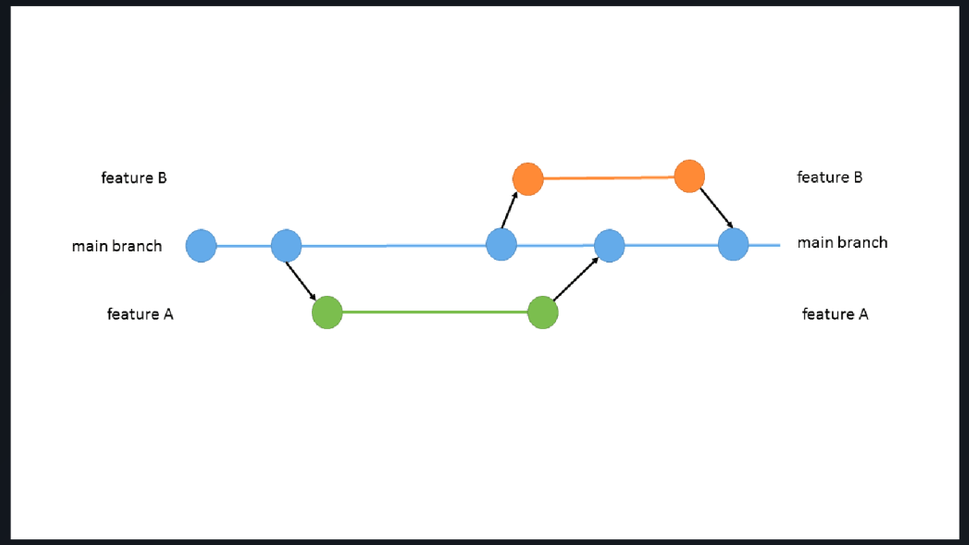
```
git branch nombre-rama
```

Cambiar de rama

```
git checkout nombre
```

Crear y cambiar en un paso:

```
git checkout -b nombre-rama
```



Comandos Básicos I

git init

Inicializa un repositorio Git en el directorio actual

- Crea carpeta .git
- Rama por defecto: main

```
git init
```

git add

Agrega archivos al Staging Area

Archivo específico

```
git add archivo.txt
```

Todos los cambios

```
git add .
```

Working → Staging

git commit

Crea un snapshot con los cambios en Staging Area

```
git commit -m "mensaje"
```

Mensaje descriptivo

Ejemplo:

```
git commit -m "feat: Add
login feature"
```

Staging → Repository

El archivo .gitignore

¿Qué es?

Indica a Git qué archivos **NO debe rastrear**

Ubicación: raíz del proyecto

Formato

Un patrón por línea

- Archivos específicos
- Patrones con comodines
- Carpetas completas

Git ignorará estos archivos y no los incluirá en los commits

Ejemplos Comunes

`node_modules/`
Dependencias del proyecto

`.env`
Variables de entorno con credenciales

`*.log`
Archivos de log

`dist/ build/`
Carpetas de compilación

`.DS_Store`
Archivos de sistema macOS

`*.tmp`
Archivos temporales

Crear archivo:

```
touch .gitignore
```

Comandos Básicos II

git status

Muestra estado del directorio de trabajo

- Archivos modificados
- Archivos en staging
- Archivos no trackeados
- Rama actual

```
git status
```

git log

Muestra historial de commits

```
git log
```

Detallado

```
git log --oneline
```

Compacto

```
git log --oneline --graph --all
```

Con gráfico

Ejemplo de git log

Git CMD

```
D:\Git\Demo>git log --stat
commit 75de18360ec12722896679c228969ed63b521004 (HEAD -> master)
Author: Dino Esposito <dino.esposito@youbiquitous.net>
Date:   Mon Sep 7 18:48:20 2020 +0200

    Changes made at 01:06PM

 hello.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)

commit 7eb9b60e6b8a6c12535a24b878d4dc6e4091270b
Author: Dino Esposito <dino.esposito@youbiquitous.net>
Date:   Mon Sep 7 13:07:52 2020 +0200

    Signed-off-by: Dino Esposito <dino.esposito@youbiquitous.net>

 hello.txt | 1 +
 1 file changed, 1 insertion(+)

D:\Git\Demo>
```

Trabajo con Ramas I

Listar ramas

```
git branch
```

Ramas locales

```
git branch -a
```

Todas las ramas (locales + remotas)

Crear rama

```
git branch nombre-rama
```

Ejemplo: git branch feature/login

Cambiar de rama

```
git checkout nombre-rama
```

Ejemplo: git checkout feature/login

Crear y cambiar en un paso

```
git checkout -b nombre-rama
```

Ejemplo: git checkout -b feature/login

Combina crear + cambiar de rama

Ver dónde estás

```
git status
```

Trabajo con Ramas II - Merge

Merge

Fusionar ramas para integrar cambios

- Une historias de diferentes ramas
- Crea un commit de merge

Pasos para merge

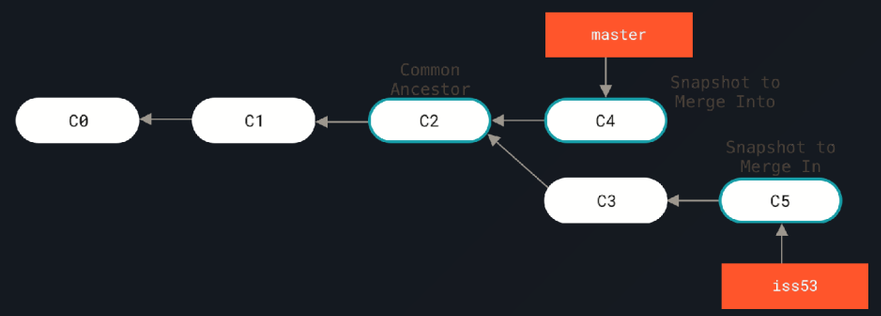
1. `git checkout main` Ir a rama destino
2. `git merge nombre-rama` Fusionar rama

Conflictos

Pueden ocurrir si hay cambios incompatibles

- Editar archivos manualmente
- `git add archivo-resuelto`
- `git commit`

Diagrama de Merge



Repositorios Remotos

¿Qué es?

Copia del repositorio en un servidor remoto

GitHub • GitLab • Bitbucket

Conceptos Importantes

origin Nombre por defecto del repositorio remoto

remote Conexión entre local y remoto

Permite colaborar con otros desarrolladores

Comandos Esenciales

Ver repositorios remotos

```
git remote -v
```

Agregar remote

```
git remote add origin https://...
```

Traer cambios (fetch + merge)

```
git pull origin main
```

Descarga y fusiona cambios del servidor

Enviar cambios

```
git push origin main
```

Sube tus commits al servidor

Flujo de Trabajo Práctico

1 Clonar o Inicializar

Clonar repositorio existente

```
git clone
https://github.com/usuario/repo.git
```

Copia completa con todo el historial

Crear repositorio nuevo

```
git init
```

Repositorio vacío desde cero

2 Crear rama para feature

```
git checkout -b feature/nueva-
funcionalidad
```

3 Hacer cambios y commits

```
git add .
git commit -m "feat: descripción"
```

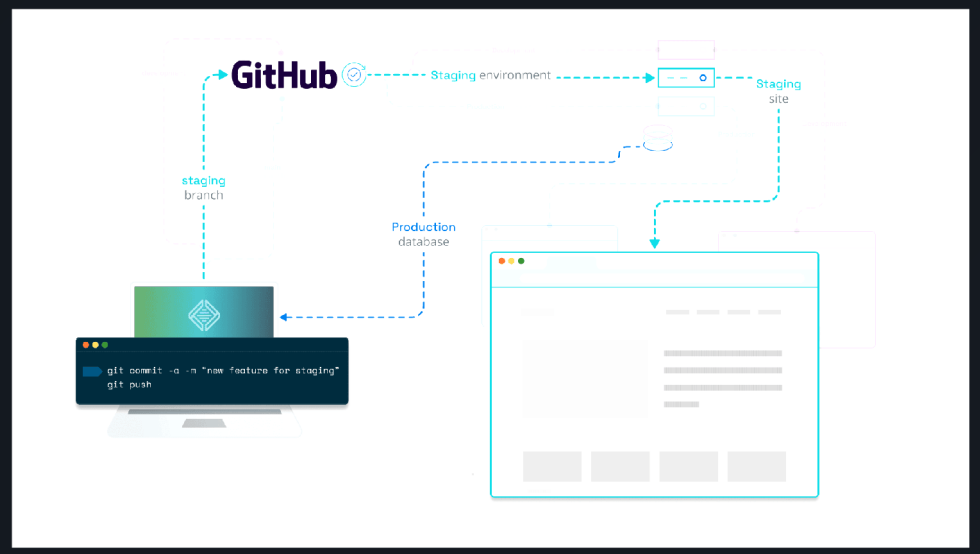
4 Fusionar con main

```
git checkout main
git merge feature/nueva-funcionalidad
```

5 Enviar al remoto

```
git push origin main
```

Diagrama del Flujo de Trabajo



Resumen y Próximos Pasos

Puntos Clave

- Git crea snapshots
- Tres estados fundamentales
- Ramas son punteros
- Commits atómicos y descriptivos

Comandos Esenciales

`init`	`add`
`commit`	`status`
`log`	`branch`
`checkout`	`merge`

Próximos Pasos

Practicar
Crear proyectos personales y experimentar con comandos

Merge vs Rebase
Entender diferencias y casos de uso

Branching Strategy
Crear estrategia de branching simple

Git Flow
Aprender flujos de trabajo avanzados

Recursos

Pro Git Book
`git-scm.com/book`
Libro oficial gratuito

GitHub Skills
Cursos interactivos gratuitos
Aprende practicando

Learn Git Branching
Herramienta visual interactiva
Visualiza branches y commits